

实验 6 - Transformer 与生成模型

数据科学与大数据技术专业

第 13–15 周

1 实验目标

本次实验围绕 Transformer、语言模型微调和扩散模型展开。与前面实验只聚焦一个模型不同，实验 6 现在包含四个相互关联但侧重点不同的实验：本地小语言模型观察、手动实现 MicroGPT、软提示微调和扩散模型采样。完成本实验后，同学们应能够：

- 理解 tokenizer、chat template、next-token probability 和采样策略在本地语言模型推理中的作用。
- 从零实现 causal mask 与 scaled dot-product attention，能够解释 decoder-only Transformer 为什么不能看到未来 token。
- 使用纯 Python 手动实现一个字符级 MicroGPT，理解自动微分、RMSNorm、多头自注意力、KV cache、交叉熵损失和 Adam 更新。
- 理解监督微调中的样本模板、label masking 和参数高效微调思想，并实现 soft prompt tuning 的前向传播与训练 step。
- 理解扩散模型的正向加噪、噪声预测目标和反向去噪采样流程，能够比较采样步数、随机种子和 scheduler 对生成结果的影响。
- 能够通过测试结果、损失曲线、生成样例和调试记录说明自己的实现是否正确，并规范记录 AI 辅助学习过程。

2 实验环境

- 操作系统: Windows / Linux / macOS
- Python 3.8 及以上
- 推荐开发环境: Jupyter Notebook 或 VS Code
- 基础依赖: `math`、`random`、`os`、`pathlib`
- 主要第三方依赖: `torch`、`transformers`、`diffusers`、`numpy`、`matplotlib`、`Pillow`
- 实验文件:
 - 2025-实验 6-1.ipynb: Transformer 与本地小语言模型。
 - 2025-实验 6.ipynb: Transformer 与手动实现 MicroGPT。

- 2025-实验 6-2-模型微调实验.ipynb: 小语言模型微调实验.
- 2025-实验 6-3-扩散模型实验.ipynb: 扩散模型教学实验.
- input.txt: 字符级人名生成数据; 若目录中缺少该文件, Notebook 会在运行时自动下载.

如本地环境缺少依赖, 可在终端中执行:

```
pip install torch transformers diffusers numpy matplotlib pillow jupyter ipykernel
```

若使用 Conda, 可先创建或进入课程实验环境, 再安装上述包. `torch` 的安装方式会随操作系统、CPU/GPU 和 CUDA 版本变化; 若默认安装不可用, 请按本机硬件环境选择合适版本. 本实验可以在 CPU 上完成, 但本地小语言模型、微调和扩散模型部分运行会明显慢于 GPU.

3 背景知识

3.1 从 Transformer 到生成模型实验

Transformer 的核心思想是用注意力机制在序列内部建立依赖关系. 对语言模型而言, 最常见的训练目标是 next-token prediction: 给定前面的 token, 预测下一个 token. 若序列为 $x_1, x_2, \dots, x_T$, 自回归语言模型把联合概率分解为:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}). \quad (3.1)$$

GPT 属于 decoder-only Transformer. 对位置 t , 模型只能使用 $x_{\leq t}$ 的信息预测后续 token, 不能使用未来 token. 这个约束在训练中通常由 causal mask 实现, 在逐 token 推理中也可以由 KV cache 的可见历史自然保证.

实验 6 的四个 notebook 对应四个学习层次:

1. **观察真实小模型:** 使用 `transformers` 加载本地小语言模型, 观察 tokenizer、chat template、注意力 mask、候选 token 和采样策略.
2. **手动实现核心机制:** 不依赖深度学习框架, 用纯 Python 写出自动微分和极简 GPT, 看清每一步梯度和注意力计算.
3. **改变模型行为:** 使用少量问答样本和 soft prompt tuning, 观察微调前后输出格式与内容是否发生变化.
4. **扩展到图像生成:** 用扩散模型理解另一类生成范式, 对比语言模型的逐 token 生成与扩散模型的逐步去噪.

3.2 Tokenizer、Chat Template 与采样

语言模型真正接收的是 token id 序列, 而不是屏幕上的自然语言字符串. Tokenizer 将文本切分成词表中的 token, 再映射为整数 id. 同一个英文单词是否包含前导空格、一个中文词被拆成几个 token、代码中的缩进和换行如何编码, 都可能影响模型看到的输入.

指令模型还需要 chat template. 用户写下的问题会被包装成带有 system、user、assistant 等角色标记的模型输入. 如果 template 与模型训练时的格式不匹配, 模型可能仍然输出文字, 但更容易忽略角色约束、复读 prompt 或把问答任务当作普通续写.

每一步语言模型会输出整个词表上的 logits. 经过 Softmax 后得到概率:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}. \quad (3.2)$$

生成时可以直接选择最大概率 token, 也可以进行带随机性的采样. Temperature 控制分布尖锐程度, top_k 只保留概率最高的 k 个候选, top_p 保留累计概率达到 p 的最小候选集合. 这些策略不会改变模型参数, 改变的是从模型概率分布中选择 token 的方式.

3.3 因果自注意力

Self-attention 先把每个位置的隐藏向量线性变换为 Query、Key、Value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (3.3)$$

缩放点积注意力为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V. \quad (3.4)$$

其中 M 是 mask. 对 decoder-only 模型, 位置 i 只能关注 $j \leq i$ 的位置, 因此未来位置会被 mask 为一个极小值, Softmax 后概率接近 0. 除以 $\sqrt{d_k}$ 是为了控制点积尺度, 避免维度较大时 logits 过大, Softmax 过早饱和.

多头注意力把隐藏维度拆成多个子空间, 每个头独立计算注意力, 再拼接并做输出投影. 从工程角度看, 多头不是简单重复计算, 而是让模型可以在不同子空间中关注不同关系, 例如局部词法关系、长距离引用或代码括号匹配.

3.4 手动 MicroGPT 与自动微分

手动实现 MicroGPT 的目的不是追求模型效果, 而是拆开深度学习框架隐藏的机制. 自动微分系统在前向传播时记录计算图, 在反向传播时按链式法则累加梯度. 本实验的 Value 类处理标量节点, 其中:

- **data**: 当前节点的标量值.
- **grad**: 损失函数对当前节点的梯度.
- **_children**: 产生当前节点的输入节点.
- **_local_grads**: 当前节点对每个输入节点的局部导数.

例如 $y = a \cdot b$ 的局部导数为:

$$\frac{\partial y}{\partial a} = b, \quad \frac{\partial y}{\partial b} = a. \quad (3.5)$$

若上游梯度为 $\partial L / \partial y$, 则反向传播时需要累加:

$$\frac{\partial L}{\partial a} += \frac{\partial L}{\partial y} \frac{\partial y}{\partial a}. \quad (3.6)$$

MicroGPT 使用字符级 tokenizer. 对一个名字 `emma`, 训练序列可以写为:

$$[\text{BOS}, e, m, m, a, \text{BOS}]. \quad (3.7)$$

模型在每个位置预测下一个 token, 损失使用负对数似然:

$$\mathcal{L}_t = -\log P(x_{t+1} | x_{\leq t}). \quad (3.8)$$

实验中采用 RMSNorm 稳定激活尺度:

$$\text{RMSNorm}(x_i) = \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}}, \quad \epsilon = 10^{-5}. \quad (3.9)$$

3.5 微调与 Soft Prompt

监督微调不是从零教会模型语言, 而是用任务样本告诉模型在某种输入格式下应该如何回答. 对问答任务, 常见样本格式为:

Question: ...

Answer: ...

训练 causal language model 时, 通常只希望答案部分参与损失, 因此会把 prompt 部分的 label 设为 -100. 在 Hugging Face 的语言模型损失中, -100 表示该位置不计入 loss. 这一步称为 label masking.

全量微调会更新模型所有参数, 成本高且小数据下容易过拟合. 本实验使用 soft prompt tuning: 冻结原模型, 只在输入 embedding 前拼接若干可学习向量:

$$E' = [p_1, p_2, \dots, p_m, e_1, e_2, \dots, e_n]. \quad (3.10)$$

这些 p_i 不是人类可读 token, 但可以通过反向传播学习如何引导模型输出更符合任务的回答. 因为可训练参数很少, soft prompt tuning 适合课堂上观察微调现象, 但它的能力也受模型规模、样本数量和训练步数限制.

3.6 扩散模型

扩散模型使用逐步去噪的方式生成图像. DDPM 的正向过程是人为规定的加噪过程. 给定干净图片 x_0 和噪声 $\epsilon \sim \mathcal{N}(0, I)$, 第 t 步带噪图片可直接写为:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (3.11)$$

t 越大, $\bar{\alpha}_t$ 越小, 图像中保留的原始信号越少.

训练时, U-Net 接收 x_t 和时间步 t , 预测噪声:

$$\mathcal{L} = \|\epsilon_\theta(x_t, t) - \epsilon\|_2^2. \quad (3.12)$$

采样时从纯噪声开始, 反复用 U-Net 预测噪声, 再由 scheduler 计算上一个时间步的图片. DDPM 通常步数多、结果稳定但较慢; DDIM 可以用较少步数加速采样, 但步数过少时图像质量可能下降.

4 实验步骤

实验 6.1: Transformer 与本地小语言模型 (2025-实验 6-1.ipynb)

实验目标: 加载一个本地小语言模型, 观察文本进入模型前后的变化, 并手动实现若干推理关键环节. 本实验重点不是训练模型, 而是看清 tokenizer、chat template、attention、logits 和采样策略如何共同决定输出.

6.1.1 Tokenizer 与 Chat Template

练习 1: 写一个 token 观察器

实现要点:

- 调用 tokenizer 将输入文本编码为 `input_ids`.
- 将每个 token id 转回 tokenizer 内部 token 字符串.
- 对比中文、英文、空格、标点和代码片段的切分方式.

分析时不要只写“token 数不同”, 应说明模型实际看到的是整数序列, 不是自然语言词语本身. 英文中的前导空格、中文词语拆分和代码缩进都可能改变 token 序列.

练习 2: 把 messages 变成模型输入

实现要点:

- 使用 `tokenizer.apply_chat_template` 将 system/user/assistant 消息包装成模型训练时熟悉的格式.
- 再调用 tokenizer 生成 `input_ids` 和 `attention_mask`.
- Decode 后观察原始用户问题前后增加了哪些特殊标记.

报告中应解释 chat template 为什么会影响指令模型行为. 如果模板错误, 模型可能把问题当成普通续写, 而不是进入“回答用户”的模式.

6.1.2 因果注意力

练习 3: 实现 causal mask 和 scaled dot-product attention

实现要点:

- 构造下三角 causal mask, 使位置 i 只能看到 $0, \dots, i$.
- 计算注意力分数 `scores = q @ k.transpose(-2, -1) / sqrt(d_k)`.
- 对未来位置填入极小值或 `-inf`, 再对最后一个维度做 Softmax.
- 用注意力权重乘以 V 得到输出.

完成后应检查未来位置的注意力权重是否为 0, 每一行注意力权重是否归一化为 1. 需要特别注意 mask 的形状要能广播到 batch 和 head 维度.

6.1.3 Next-token Probability 与采样策略

练习 4: 实现候选 token 查看器

实现要点:

- 对 prompt 做模型前向传播.
- 取最后一个位置的 logits, 而不是所有位置的 logits.
- 使用 Softmax 得到概率分布, 再用 `topk` 找出概率最高的候选 token.

分析时应说明 logits 是未归一化分数, 只有经过 Softmax 才能解释为概率. 同一个 prompt 改动一个词, 下一步候选分布也可能明显变化.

练习 5: 实现 logits 过滤器

实现要点:

- 复制传入的 logits, 避免原地修改导致后续调试困难.
- 先应用 temperature, 再应用 `top_k` 和 `top_p`.
- 被屏蔽的 token 位置应设为 `-inf`, 使 Softmax 后概率为 0.

`top_k` 是固定保留 k 个候选, `top_p` 是根据累计概率动态决定候选集合大小. 报告中应比较二者的差异, 不要只列出代码.

练习 6: 实现自回归生成

实现要点:

1. 将当前上下文输入模型.
2. 取最后位置 logits.
3. 调用 `filter_logits` 和采样函数得到 `next_token`.
4. 将新 token 拼接到序列末尾.
5. 遇到结束 token 或达到最大长度时停止.

请用同一个 prompt 对比 greedy、低 temperature、高 temperature、`top_k` 和 `top_p` 设置. 分析重点是输出稳定性、多样性、重复和跑题现象.

6.1.4 续写打分与 Prompt 分析

练习 7: 实现续写打分

实现要点:

- 分别编码 prompt 和 prompt+continuation.
- 构造 labels, 并将 prompt 部分设为 `-100`.
- 调用模型计算 continuation 部分的平均负对数似然.
- 将 NLL 转换为 perplexity 时使用 $PPL = \exp(NLL)$.

该练习说明语言模型可以比较两个候选续写哪个更符合它学到的分布. 但 NLL 不是事实正确性的直接证明, 它只表示模型在当前上下文下对该续写的概率偏好.

实验 6.2: Transformer 与手动实现 MicroGPT(2025-实验 6.ipynb)

实验目标: 使用纯 Python 手动实现一个字符级 GPT 风格语言模型。本实验不依赖 PyTorch 自动求导, 重点是理解计算图、局部导数、向量组件、多头注意力、训练循环和自回归采样。

6.2.1 字符级 Tokenizer 与自动微分

练习 1: 构建字符级 Tokenizer

实现要点:

- 使用 `''.join(docs)` 将所有名字拼成一个字符串。
- 使用 `sorted(set(...))` 得到唯一字符列表 `uchars`。
- 设置 `BOS = len(uchars)`。
- 设置 `vocab_size = len(uchars) + 1`。

完成后测试应显示普通字符数为 26, BOS 为 26, `vocab_size` 为 27。这里的 BOS 同时作为起始和结束 token, 因此名字序列前后都要添加它。

练习 2: 实现 `Value.__mul__` 与 `Value.__pow__`

实现要点:

- 乘法输出值为 $a \cdot b$, 子节点为 `(self, other)`, 局部导数分别为 b 和 a 。
- 幂次 x^k 的输出值为 x^k , 子节点为 `(self,)`, 局部导数为 kx^{k-1} 。
- 反向传播时梯度要累加, 不能覆盖已有梯度。

用

$$L = (a \cdot b + c)^2, \quad a = 2, b = 3, c = 1 \quad (4.1)$$

测试反向传播. 正确结果应为 $L = 49$, $a.grad = 42$, $b.grad = 28$, $c.grad = 14$ 。

6.2.2 模型参数与基础组件

本实验采用轻量配置:

含义	变量名	取值
Transformer 层数	<code>n_layer</code>	1
嵌入维度	<code>n_embd</code>	16
最大序列长度	<code>block_size</code>	16
注意力头数	<code>n_head</code>	4
每头维度	<code>head_dim</code>	4

练习 3: 实现 `linear`

函数 `linear(x, w)` 实现无偏置线性变换:

$$y_i = \sum_j W_{ij} x_j. \quad (4.2)$$

实现时外层遍历权重矩阵每一行, 内层用 `sum(wi * xi for wi, xi in zip(wo, x))` 计算点积. 返回值必须仍然是由 `Value` 组成的向量, 否则后续无法反向传播.

练习 4: 实现 `rmsnorm`

函数 `rmsnorm(x)` 对一个 `Value` 向量做 RMSNorm:

1. 计算 `ms = sum(xi * xi for xi in x) / len(x)`.
2. 计算 `scale = (ms + 1e-5) ** -0.5`.
3. 返回 `[xi * scale for xi in x]`.

完成后运行测试单元, 检查 `linear` 输出长度和值是否正确, RMSNorm 后向量 RMS 是否接近 1.

6.2.3 GPT 前向传播

练习 5: 实现 `gpt` 前向传播函数

单步前向传播的整体结构为:

1. 取出当前 token embedding 和 position embedding 并相加.
2. 对输入做 RMSNorm.
3. 对每层 Transformer 计算 Q/K/V.
4. 将当前 key 和 value 追加到缓存中.
5. 对每个注意力头计算缩放点积注意力.
6. 拼接所有头输出并做输出投影.
7. 加入残差连接.
8. 再经过 RMSNorm、MLP 和第二个残差连接.
9. 通过 `lm_head` 得到下一个 token 的 logits.

对每个头 h , 令 $hs = h * head_dim$. 需要完成:

- 从 `q` 中取当前头切片 `q_h`.
- 从历史 `keys[li]` 中取所有 `k_h`.
- 从历史 `values[li]` 中取所有 `v_h`.
- 计算 $\sum_i q_i k_i / \sqrt{head_dim}$.
- 对注意力得分调用 `softmax`.
- 对历史 value 加权求和, 得到当前头输出.
- 将所有头输出拼接成长度为 `n_embd` 的向量.

这里的因果约束来自逐 token 调用 `gpt`: 当前步只能访问已经写入缓存的历史 key/value. 若输出长度、缓存长度或 `Value` 类型检查失败, 优先检查头切片边界和缓存追加位置.

6.2.4 训练循环与自回归生成

练习 6: 实现训练循环中的损失计算

每个训练 step 包含:

1. 随机抽取一个名字 doc.
2. 编码为 [BOS] + tokens + [BOS].
3. 清空各层 keys 和 values 缓存.
4. 对每个位置调用 gpt, 得到预测下一个 token 的 logits.
5. 调用 softmax(logits) 得到概率分布.
6. 取目标 token 概率并计算 `-probs[target_id].log()`.
7. 对所有位置损失求平均, 调用 `loss.backward()`.
8. 使用 Adam 更新参数并清零梯度.

训练中应记录损失是否大体下降. 由于模型很小、数据简单、训练步数有限, 生成结果不必完美, 但应该逐渐更像英文人名.

练习 7: 实现自回归推理

生成流程为:

1. 从 `token_id = BOS` 开始.
2. 每一步调用 `gpt(token_id, pos_id, keys, values)`.
3. 将 logits 除以 temperature 后做 Softmax.
4. 使用 `random.choices` 按概率采样下一个 token.
5. 若采样到 BOS, 结束当前名字; 否则解码字符并追加到输出.

请尝试至少两个 temperature, 例如 `temperature=0.3` 和 `temperature=0.8`, 对比生成结果的保守程度和多样性.

实验 6.3: 小语言模型微调实验 (2025-实验 6-2-模型微调实验.ipynb)

实验目标: 理解微调如何改变模型在特定任务格式下的回答方式. 本实验使用少量课程问答样本和 soft prompt tuning, 只训练一小段可学习提示向量, 不全量更新基础模型参数.

6.3.1 微调前输出与训练样本构造

运行微调前生成单元, 观察基础模型是否能按照课程问答格式回答. 需要区分“模型能续写英语”和“模型会按课程助教风格回答”这两件事.

练习 1: 把问答样本变成训练样本

实现要点:

- 构造 prompt, 例如 `Question: ... Answer:.`
- 构造完整文本, 将目标答案接在 prompt 后面.

- Tokenize 完整文本, 得到 `input_ids` 和 `attention_mask`.
- 计算 prompt 的 token 长度, 并把 labels 中 prompt 部分设为 -100.

报告中应说明为什么不应该让 prompt 部分参与 loss. 本实验希望模型学习如何写答案, 而不是重复问题模板.

6.3.2 Soft Prompt 前向传播

练习 2: 实现 soft prompt 前向传播

实现要点:

- 从冻结的基础模型中取出原始 token embedding.
- 将可学习的 `soft_prompt` 扩展到 batch 维度.
- 在原始输入 embedding 前拼接 soft prompt embedding.
- 同步扩展 `attention_mask` 和 labels, 使新增 soft prompt 位置不计入 loss.
- 使用 `inputs_embeds` 调用基础模型前向传播.

需要注意, soft prompt 是连续向量, 不是 tokenizer 词表中的离散 token. 它不能直接 decode 成自然语言, 但可以通过梯度学习到有用的引导方向.

6.3.3 训练 Step 与微调后比较

练习 3: 补全一次训练 step

实现要点:

1. 调用 prompt model 前向传播.
2. 取出 loss.
3. 调用 `loss.backward()`.
4. 执行 `optimizer.step()`.
5. 执行 `optimizer.zero_grad()`.

训练后需要比较微调前后的回答. 分析时至少关注三点: loss 是否下降, 已见过问题是否更接近训练答案, 未见过问题是否也更符合目标格式. 如果模型只是记住少量样本, 这不等于它真正理解了概念.

实验 6.4: 扩散模型教学实验 (2025-实验 6-3-扩散模型实验.ipynb)

实验目标: 使用预训练扩散模型和手写的小步骤理解图像生成中的正向加噪、噪声预测和反向采样. 本实验不要求重新训练扩散模型, 重点是理解 scheduler 与 U-Net 在采样循环中的分工.

6.4.1 正向扩散

练习 1: 手写一次正向加噪

实现要点:

- 从 scheduler 中取出当前时间步对应的 $\bar{\alpha}_t$.
- 计算信号系数 $\sqrt{\bar{\alpha}_t}$.
- 计算噪声系数 $\sqrt{1 - \bar{\alpha}_t}$.
- 返回信号项与噪声项的加权和.

请观察不同 timestep 下图像从清晰到噪声的变化. t 越大, 图像中原始信号越少, 噪声越强.

6.4.2 噪声预测目标

练习 2: 计算一次噪声预测 loss

实现要点:

1. 随机抽取 timestep.
2. 使用正向公式把干净图像加噪得到 x_t .
3. 调用 U-Net 预测噪声 `noise_pred`.
4. 使用 MSE 计算 `noise_pred` 与真实噪声的差异.

该 loss 对应扩散模型训练时的基本目标. 课堂实验中只计算一次 loss, 不重新训练模型, 因为训练高质量扩散模型需要大量数据和计算资源.

6.4.3 反向去噪采样

练习 3: 手写一个最小反向去噪循环

实现要点:

- 从随机高斯噪声图片开始.
- 对每个时间步调用 U-Net 预测噪声.
- 调用 `scheduler.step` 计算上一个时间步的图片.
- 将返回结果作为下一轮输入, 直到采样结束.

请比较 DDPM 和 DDIM, 以及不同采样步数和随机种子下的图像差异. 步数更少通常更快, 但可能牺牲细节和稳定性.

6.4.4 小实验与结果记录

请至少完成一个小实验:

- 固定随机种子, 改变采样步数, 比较图像质量和耗时.
- 固定步数, 改变随机种子, 比较生成多样性.

- 固定初始条件, 比较 DDPM 与 DDIM scheduler 的输出差异.

报告中应包含参数设置、观察到的图像现象和简短解释. 不要只贴图, 需要说明这些差异和反向去噪过程之间的关系.

5 核心函数总结

实验	函数或变量	重点
6.1	<code>token_table</code> , <code>build_chat_inputs</code>	观察文本如何变成 token id, 以及 chat template 如何包装用户问题.
6.1	<code>make_causal_mask</code> , <code>scaled_dot_product_attention</code>	实现因果 mask、缩放点积、Softmax 和注意力加权求和.
6.1	<code>next_token_candidates</code> , <code>filter_logits</code> , <code>generate_with_controls</code>	观察 logits、候选 token、temperature、top_k、top_p 和自回归生成.
6.1	<code>score_continuation</code>	用 NLL/PPL 比较候选续写, 并理解概率偏好不等于事实正确.
6.2	<code>uchars</code> , <code>BOS</code> , <code>vocab_size</code>	定义字符级词表和特殊起止 token.
6.2	<code>Value.__mul__</code> , <code>Value.__pow__</code> , <code>Value.backward()</code>	建立标量计算图, 记录局部导数并反向累加梯度.
6.2	<code>linear</code> , <code>rmsnorm</code> , <code>softmax</code>	实现 GPT 中的基础向量组件和概率归一化.
6.2	<code>gpt</code> , <code>keys</code> , <code>values</code>	实现单步 GPT 前向传播和 KV cache 因果注意力.
6.3	<code>encode_example</code> , <code>collate_batch</code>	将问答样本变成 causal LM 训练样本, 并正确设置 label masking.
6.3	<code>SoftPromptModel.forward</code>	拼接可学习 soft prompt embedding, 扩展 mask 和 labels.
6.3	训练 step	前向传播、反向传播、优化器更新和清空梯度.
6.4	<code>manual_add_noise</code>	根据 $\bar{\alpha}_t$ 手写正向扩散加噪.
6.4	噪声预测 loss	调用 U-Net 预测噪声并用 MSE 对齐真实噪声.
6.4	<code>manual_sample</code> , <code>scheduler.step</code>	从随机噪声开始逐步反向去噪, 比较 DDPM/DDIM 和采样步数.

6 实验作业

1. 完成 2025-实验 6-1.ipynb, 运行本地小语言模型, 补全 tokenizer 观察器、chat template 输入、causal attention、候选 token 查看器、logits 过滤器、自回归生成和续写打分.
2. 完成 2025-实验 6.ipynb, 补全字符级 tokenizer、Value 自动微分、linear、rmsnorm、gpt 前向传播、训练损失计算和自回归推理.
3. 完成 2025-实验 6-2-模型微调实验.ipynb, 补全问答样本编码、soft prompt 前向传播和训练 step, 并比较微调前后的输出.
4. 完成 2025-实验 6-3-扩散模型实验.ipynb, 补全正向加噪、噪声预测 loss 和反向去噪循环, 并完成至少一个采样小实验.

5. 至少选择两个实验结果进行解释: 例如 token 切分表、注意力权重矩阵、MicroGPT 损失曲线、生成名字样例、微调前后回答对比、扩散采样图像对比.
6. 在实验过程记录中写明遇到的主要报错或异常现象, 以及你如何判断问题来自维度、mask、梯度、采样参数、显存/内存或模型文件加载.
7. **选做题:** 从下面题目中任选 1-2 个完成, 在 Notebook 或实验记录中保留设置、结果和简短分析.
 - **Tokenizer 侦探:** 设计 5 组容易被 tokenizer 拆分出意外结果的文本, 例如中英混排、空格变化、代码缩进、罕见符号或相似拼写, 分析 token 数量和模型候选输出如何变化.
 - **注意力可视化:** 选择两段不同 prompt, 可视化或打印 causal attention 权重矩阵, 说明某些位置为什么更关注前文中的特定 token.
 - **采样控制实验:** 在自回归生成中加入 repetition penalty、禁止连续重复字符或 no-repeat bigram 规则, 对比修改前后的生成文本或人名样例.
 - **MicroGPT 消融实验:** 分别去掉 RMSNorm、残差连接或改变 `n_head/n_embd`, 观察训练损失、生成质量和运行速度的变化.
 - **微调泛化挑战:** 自行增加 5-10 条课程问答样本, 比较微调模型在已见问题和未见问题上的回答差异, 判断它是在学习格式、记忆答案还是有一定泛化.
 - **扩散过程翻页:** 保存同一次采样中的若干中间去噪结果, 做成网格图或逐步记录, 比较 DDPM/DDIM、采样步数和随机种子对图像演化过程的影响.

7 提交要求

提交前请逐项检查材料是否齐全. 本次实验提交内容包括四个 Notebook 源文件和实验过程记录. 本课程作业上传只接受文本文件, 不接受 PDF、Word 文档、图片、截图、压缩包等二进制文件. 采用这一规则, 一方面是为了引入 AI 辅助评阅, 另一方面也是希望大家养成用清晰文本表达实验过程和结论的习惯.

1. Notebook 文件

- 提交补全后的四个 Notebook:
 - 2025-实验 6-1.ipynb
 - 2025-实验 6.ipynb
 - 2025-实验 6-2-模型微调实验.ipynb
 - 2025-实验 6-3-扩散模型实验.ipynb
- Notebook 本身属于文本格式, 可以上传. 请保留必要的测试输出、损失曲线、候选 token 表格、生成样例、微调前后对比和扩散模型采样结果.
- 如果实验报告必须包含配图, 请将图片保留在 Notebook 输出中, 形成完整实验报告; 不要单独上传图片、PDF 或压缩包.
- 若完成选做内容, 请在 Notebook 或实验记录中保留配置、指标、结果和简要说明.

2. 实验过程与 AI 互动记录文本文件 (.md / .txt)

- 必须额外提交一份文本文件, 作为实验过程与 AI 互动记录, 推荐使用 `.md` 或 `.txt`.
- 记录内容至少包括: 遇到的主要问题、查阅资料或询问 AI 的问题、自己的判断、调试尝试、修改思路和实验结论.
- 不要把 AI 的回复原文放入记录中, 也不要复制粘贴大段代码; 记录应只包含你自己写的内容.
- 若使用 $\text{\LaTeX}$ 、Word 或其他文档工具撰写报告, 请使用 `pandoc` 等工具转换为纯文本或 Markdown 后提交.
- 建议文件命名为 AI 问答记录 `.md` 或 实验过程记录 `.md`.

8 关于作业的重要说明

本实验覆盖面较广, 既有底层机制手写, 也有真实模型调用. 完成作业时, 不仅要让测试单元通过, 还要能解释为什么这样实现. 对 Transformer 部分, 优先检查 token 序列、mask 方向、logits 位置和采样参数; 对 MicroGPT 部分, 优先检查 `Value` 运算、向量长度、KV cache 和梯度累加; 对微调部分, 优先检查 label masking、soft prompt 拼接和优化器参数; 对扩散模型部分, 优先检查 timestep、张量形状、scheduler 调用顺序和设备位置.

对于必做内容, 可以使用 AI 工具辅助理解概念、解释报错、查询 API 用法或检查维度, 但不得直接让 AI 代写全部必做代码后不加理解地提交. 若使用 AI 工具, 必须在文本记录中如实写明自己的提问、自己的判断和后续修改过程.

AI 互动记录应只保留学生自己写的内容. 不要粘贴 AI 的完整回复, 不要粘贴复制来的大段代码, 也不要把聊天记录原样导出后提交. 记录的重点是说明你如何提出问题、如何判断回答是否可靠, 以及如何把这些信息转化为自己的实现和分析.

选做内容可以更自由地使用 AI 工具, 但仍建议记录关键尝试过程和实验结论.